

Fine-tuning, Ecosistema, APIs, costos y Huggingface

Mg. Ing. Abraham Rodriguez · FIUBA

Programa de la materia

- Repaso de Transformers, Arquitectura y Tokenizers.
- Arquitecturas de LLMs, Transformer Decoder.
- MoEs, técnicas de prompts, intro evaluación de LLMs.
- **Fine-tuning, Ecosistema actual, APIs, costos, HuggingFace y OpenAI.**
- Modelos locales, servidores de inferencia, Reinforcement Learning.
- Modelos de Embeddings, vector DBs, naïve RAG, chatbots.
- Intro a Agentes, LLMs de razonamiento.
- Multimodalidad y estado del arte.

Mapa del ecosistema

Tres capas distintas: **quien entrena**, **quien publica pesos**, **quien vende computo**.

Pesos cerrados (solo API)	Pesos abiertos	Infraestructura
OpenAI (GPT-5.x)	Llama (Meta)	Together, Fireworks, Groq
Anthropic (Claude)	Qwen (Alibaba)	OpenRouter (router multi-proveedor), Cloudflare Workers AI
Google (Gemini)	DeepSeek	AWS Bedrock, GC Vertex, Azure OpenAI
X.AI (Grok)	Mistral, GLM, Gemma	HuggingFace Inference Providers

Pesos abiertos no es lo mismo que open source

- **Apache 2.0 / MIT** (Qwen, DeepSeek, varios Mistral): uso libre, incluso comercial
- **Licencias "community"** (Llama): libre hasta cierto umbral de usuarios, cláusulas de uso aceptable, restricciones de nombre para modelos derivados
- **Gated repos** (Llama, Gemma): hay que aceptar términos en el Hub para descargar
- Los **datos de entrenamiento** muchas veces **no son publicados** osea son open weights, pero no open source

In-house vs Cloud vs Edge

	In-house	Cloud / API	Edge (dispositivo)
Expertise técnico	Alto	Bajo	Alto
Costo inicial	Alto	Casi nulo	Medio
Costo total a escala	Bajo	Alto	Bajo
Escalabilidad	Limitada por el hardware	Elastica	Por dispositivo
Control del dato	Total	Del proveedor	Total, sin red
Personalización	Total (fine-tuning propio)	La que exponga la API	Total, con modelos chicos
Riesgo de caída	Tuyo	Del proveedor	Bajo

Verificar contra la tabla original del deck viejo (slide 10) antes de proyectar.

Escala: lo que importa ya no es el tamaño total

- 2023: el número que importaba era **parámetros totales**
- 2026: con MoE el número que importa es **parámetros activos por token**
- Un MoE de 600B totales con 30B activos cuesta como un denso de 30B en **computo**, pero como uno de 600B en **memoria**
- Por eso el precio por token de un MoE no se deduce de su tamaño nominal

Leaderboards: donde mirar y por que desconfiar

Dónde mirar

- **LMarena** — preferencia humana, pareado y ciego
- **Artificial Analysis** — calidad vs precio vs latencia
- **HF Open LLM Leaderboard** — solo pesos abiertos, reproducible
- **SWE-bench, GPQA, AIME** — especificos

Por qué desconfiar

- **Contaminacion** — el benchmark puede estar en el set de entrenamiento
- **Overfitting al leaderboard** — se optimiza la métrica, no la capacidad
- **Sesgo de estilo** — en arenas gana la respuesta larga y bien formateada

Qué es el Hub

- **Models** (~2M repos), **Datasets**, **Spaces** (demos Gradio / Streamlit)
- Cada repo es **un repo git** con LFS. git clone funciona
- El SDK: `huggingface_hub · transformers · datasets · tokenizers`
- `revision=` es un commit hash: pinarlo es lo que hace reproducible un experimento

Anatomía de un repo de modelo

- **config.json** — la arquitectura. Ahi estan n_layer, n_head, n_embd del TP-1
- **model.safetensors** — los pesos
- **tokenizer.json** — el tokenizer (clase 1)
- **README.md** — la model card: licencia, datos, límites, evaluaciones

safetensors vs pickle

- .bin / .pt de PyTorch = **pickle** = ejecución arbitraria de código al cargar
- **safetensors** = formato de solo datos, sin código, con carga zero-copy
- Regla práctica: si un repo solo ofrece .bin, es señal de que está sin mantener

La libreria transformers

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tok = AutoTokenizer.from_pretrained("modelo")
model = AutoModelForCausalLM.from_pretrained("modelo")

out = model.generate(**tok("hola", return_tensors="pt"),
                     do_sample=True, temperature=0.8, top_k=50, top_p=0.95)
```

- AutoModelFor... elige la clase leyendo **config.json**
- pipeline() para prototipos; el modelo crudo para todo lo demas

El stack de entrenamiento: PEFT, TRL, Accelerate

- **peft** — LoRA, QLoRA, adapters ← el TP-2 es implementar esto a mano
- **trl** — SFTTrainer, DPO, GRPO
- **accelerate** — multi-GPU y offloading sin reescribir el loop
- **datasets** — lazy-loading de datasets con memory-mapping

Del Hub a produccion

- **Inference Providers** — un endpoint HF que rutea a Together, Groq, etc.
- **Inference Endpoints** — tu modelo, tu GPU dedicada, por hora
- **Spaces** — demo pública gratis (con GPU, pago)
- **AutoTrain / Jobs** — fine-tuning gestionado
- Alternativa: te lo bajás y lo servís vos → **clase 5**

APIs

API OpenAI

La [API OpenAI](#) es la API que define el default para la mayoría de proveedores de LLMs, la razón es compatibilidad y facilidad de integración.

```
client.chat.completions.create(  
    model="...",  
    messages=[{"role": "system",      "content": "..."},  
              {"role": "user",       "content": "..."},  
              {"role": "assistant",  "content": "..."}],  
    temperature=0.7, top_p=0.9, max_tokens=512,  
)
```

- messages es **una lista, no un string**: el servidor le aplica el chat template
- temperature y top_p: los mismos del TP-1

El estándar de facto

- La forma chat/completions de OpenAI la copiaron **todos**
- Together, Groq, Cloudflare, etc : endpoints OpenAI-compatibles

```
# ollama, local
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

# Cloudflare Workers AI
client = OpenAI(
    base_url=f"https://api.cloudflare.com/client/v4/accounts/{ACCOUNT_ID}/ai/v1",
    api_key=CF_API_KEY)

client.chat.completions.create(model="@cf/meta/llama-3.1-8b-instruct", ...)
```

Cambiás base_url y api_key, y la aplicación sigue igual

Otros APIs

Claude cuenta con su propio formato independiente llamado messages API. Por otro lado, **Bedrock** cuenta con su propio formato, converse API.

En este caso todos los APIs son incompatibles por contrato pero la comparten los mismos conceptos.

[Claude: Messages API](#)
[Messages API Create](#)
[Claude: Reduce latency](#)

[Converse API](#)
[Inference with Converse API](#)

Sin embargo, debido a que la compatibilidad es fundamental en aplicaciones con LLMs, se suelen crear puentes entre APIs por ejemplo:

Converse API -> Middleware (Puente) -> Formato OpenAI -> Cliente

Ejemplo: <https://github.com/aws-samples/bedrock-access-gateway>

Streaming y latencia

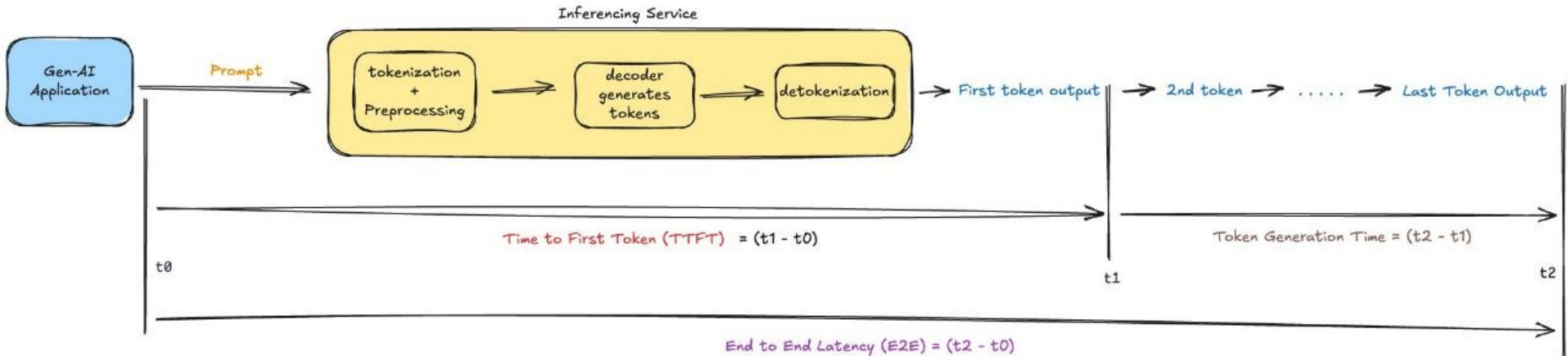
- TTFT (time to first token) — dominado por el prefill, crece con el prompt
- TPOT (time per output token) — el decode, uno por uno
- Throughput — lo que le importa al que sirve, no al usuario
- E2E (end to end) — lo que percibe el usuario: $E2E = TTFT + TPOT \times N_{ou}$
- stream=True no acelera nada: mejora el TTFT percibido

[AWS Bedrock: Latency Optimized Inference](#)

[Optimizing AI responsiveness: A practical guide to Amazon Bedrock latency-optimized inference](#)

[OpenAI: Latency Optimization](#)

Streaming y latencia



Time to First Token (TTFT) = Time to first Token - Time from request submission

Output Token Per Second (OTPS) = Total Tokens in Response / Token Generation Time

End to End Latency (E2E-Latency) = TTFT + Token Generation Time (or, $t_2 - t_0$)

Más allá del texto libre

Salidas estructuradas

- JSON mode garantiza **JSON válido**
- Structured outputs garantiza **tu esquema**
- Se implementa enmascarando logits: lo que rompe la gramática va a -inf

Tool / function calling

- Le pasás firmas de funciones
- El modelo devuelve **qué función y con qué args**.
No ejecuta nada
- **Es el 90% de lo que es un agente**

La unidad es el token

- Se factura por **token de entrada** y **token de salida**, por millón
- Regla de bolsillo: 1 token $\sim$ 4 caracteres $\sim$ 0.75 palabras en inglés
- **En español rinde peor**: los tokenizers están entrenados mayormente en inglés, el mismo texto cuesta $\sim$ 1.3-1.5x más tokens
- Operar en español cuesta más plata, y es una consecuencia del tokenizer

Por qué el output cuesta 3-5x más que el input

- **Prefill** (input): todos los tokens en paralelo, una pasada, GPU saturada
- **Decode** (output): un token por forward, secuencial, GPU subutilizada
- El precio refleja el hardware, no el margen

Órdenes de magnitud (USD por millón de tokens)

Verificar los números el día de la clase. Lo que se enseña es la estructura y los ratios, no las cifras.

Nivel	Ejemplos (sep. 2026)	Input	Output
Frontera	Claude Opus 5, GPT-5.6 tope de gama	~\$4-5	~\$20-30
Trabajo pesado	Claude Sonnet 5, Gemini 3.1 Pro	~\$2	~\$10-12
Barato y capaz	GPT-5.6 Luna, Haiku 4.5	~\$0.20-1	~\$1.20-5
Piso	Gemini Flash-Lite, DeepSeek V4	~\$0.10-0.14	~\$0.28-0.40

Fuentes: platform.openai.com/docs/pricing · anthropic.com/pricing · ai.google.dev/pricing · api-docs.deepseek.com

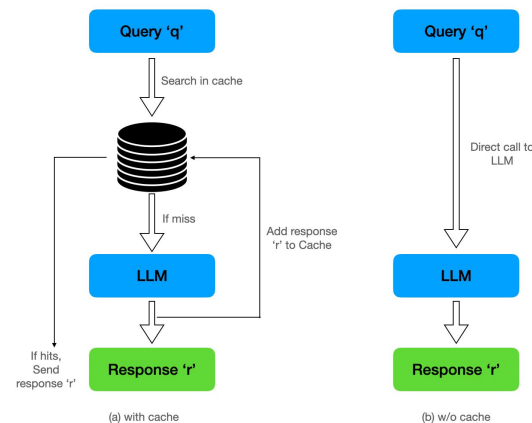
Lo que falta en esa cuenta: el historial

- Un chatbot **re-manda toda la conversacion en cada turno**
- El costo por conversacion es **cuadrático**, no lineal, en la cantidad de turnos
- El turno 8 paga los 7 anteriores otra vez
- Mitigaciones: ventana deslizando, resumen del historial, **prompt caching**

Los dos descuentos que casi nadie usa

- **Prompt caching** — el prefijo repetido (system prompt, few-shot, documento) se cachea; el hit cuesta ~10% del input. Requiere poner **lo estable al principio**
- **Batch API** — trabajo asincrónico con ventana de ~24 h: **50% de descuento**
- Combinados, un pipeline offline bien armado sale **3-5x menos** que el ingenuo

<https://developers.openai.com/api/docs/guides/prompt-caching>



Los tokens que no ves

- Los modelos de razonamiento generan **tokens de pensamiento** que se facturan como output y **no aparecen** en la respuesta
- Una respuesta de 200 tokens puede haber costado 4.000
- Controlable con el parámetro de esfuerzo o presupuesto de razonamiento, lo veremos en **clase 7**

API vs servirlo vos: el break-even

- **API:** \$/token, cero costo fijo, escala sola, sin control del dato
- **Propio:** \$/hora de GPU, se paga aunque este ocioso, control total
- Equilibrio ~ cuando el **uso sostenido** llena la GPU. Por debajo, la API gana casi siempre
- Lo que empuja a self-hosting no suele ser el precio: es **privacidad, latencia o un modelo fine-tuneado propio**

Sesgo y toxicidad: lo que viene en la caja

- **Sesgo social** — resultados desiguales entre grupos, heredados de los datos
- **Toxicidad** — capacidad de generar contenido ofensivo o dañino
- Taxonomía de daños: lenguaje denigratorio, rendimiento dispar por dialecto, borrado, normas excluyentes, tergiversación, estereotipos
- Elegir proveedor es también elegir **qué sesgos heredás** y qué política de uso aceptás

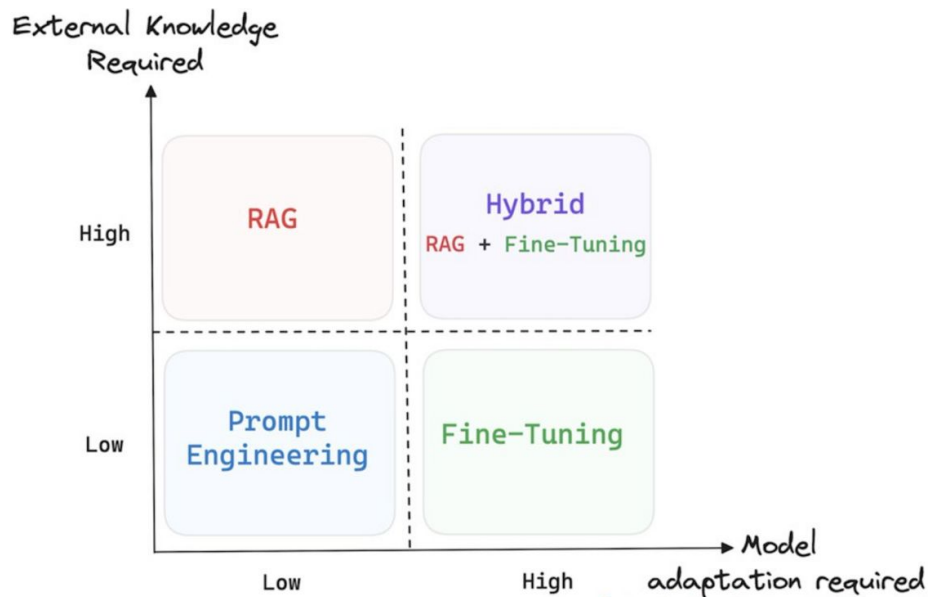
Dónde se mitiga

- **Pre-procesamiento** — datos: aumento, filtrado, reponderación
- **Durante el entrenamiento** — arquitectura, loss, **actualización selectiva de parámetros**
- **Intra-procesamiento** — decodificación restringida, redistribución de pesos
- **Post-procesamiento** — reescritura de la salida

Fine-tuning

Necesidad

La pregunta es, necesito hacer fine-tuning o solo debo de enrobustecer mi sistema?



Andrej Karpathy ✓
@karpathy

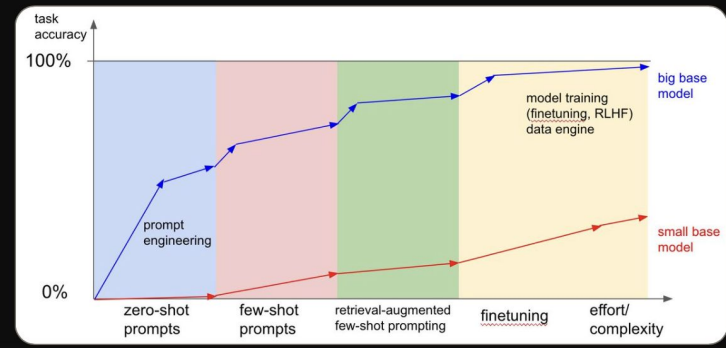
It's a great question. I roughly think of finetuning as analogous to expertise in people:

- Describe a task in words ~= zero-shot prompting
- Give examples of solving task ~= few-shot prompting
- Allow person to practice task ~= finetuning

With this analogy in mind, it's awesome that we have models that can reach high levels of accuracy across many tasks with prompting alone, but I also expect that reaching top tier performance will include finetuning, especially in applications with concrete well-defined tasks where it is possible to collect a lot of data and "practice" on it.

Rough picture to have in mind maybe. Small models are incapable of in-context learning and will benefit very little from prompt engineering, but depending on the difficulty of the task it may be possible to still finetune them into decent experts.

Big caveat all of this is still very new.



El espectro de adaptación

Ordenado por costo creciente.

Técnica	Qué cuesta	Qué arregla
Prompting / system prompt	tokens	instrucciones, tono
Few-shot	tokens en cada request	formato, estilo
RAG	infra + tokens	conocimiento que el modelo no tiene
PEFT / LoRA	horas de GPU, una vez	forma, formato, dominio, un comportamiento
Full fine-tuning	muchas horas de GPU	con más riesgo y más costo
Preentrenamiento	millones de dólares	Modelo fundacional

Qué arregla y qué NO arregla el fine-tuning

Si

- Formato de salida y esquema propio
- Tono y estilo
- Jerga de dominio
- Tareas repetitivas y acotadas
- Achicar el prompt: el comportamiento queda en los pesos

No, o mal

- Meter conocimiento nuevo y factual
- El modelo aprende **la forma** de la respuesta más rápido que el contenido
- Resultado: responde con seguridad y datos inventados
- Conocimiento que cambia (**RAG**)

Los dos ejes

¿QUÉ DATOS?

SFT (instruccion, respuesta)

|
instruction tuning

Preferencias (A mejor que B)

|
RLHF / DPO / GRPO

¿QUÉ PARÁMETROS?

Full fine-tuning

<-->

PEFT (LoRA, QLoRA, adapters)

SFT: de completar texto a seguir instrucciones

- Un modelo preentrenado **completa texto**; no sigue instrucciones
- SFT = seguir entrenando con pares (instrucción, respuesta) en formato de chat
- Es el objetivo L3 de GPT-1: el mismo next-token loss, sobre un subconjunto de posiciones

Como sabemos, dado un dataset etiquetado, realizamos SFT para ajustar a un nuevo objetivo, el cual puede ser un formato, tono, etc.

Datasets:

[Ultradata SFT](#)

[Nemotron SFT](#)

$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y)$$

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m)$$

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

El problema: la memoria

Fine-tuning es mas barato computacionalmente que pre-entrenar, pero siempre requiere un grado de computo, especialmente memoria, por ejemplo:

- Con AdamW, por cada parámetro hay que guardar:

pesos 4 B + gradientes 4 B + Adam (m, v) 8 B ~ 16 bytes/parámetro

modelo de 7B parámetros

inferencia (fp16) ~14 GB

full fine-tuning ~112 GB + activaciones

<https://apxml.com/courses/lora-peft-efficient-llm-training/chapter-1-fine-tuning-efficiency-foundations/full-fine-tuning-costs>

LoRA: la hipótesis

[Hu et al., 2021 — LoRA: Low-Rank Adaptation of Large Language Models](#)

- Antecedente: Aghajanyan et al. (2020) muestran que los modelos preentrenados tienen **dimensión intrínseca baja**: se adaptan bien optimizando en un subespacio chico
- Hipótesis de LoRA: entonces **el update también tiene rango bajo**
- No hace falta aprender un ΔW completo de $d \times k$; alcanza con su versión factorizada

Asuma

[Lora in depth](#)

<https://huggingface.co/learn/llm-course/chapter11/4>

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

ABSTRACT

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which re-trains all model parameters, becomes less feasible. Using GPT-3 175B as an example – deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times. LoRA performs on-par or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters, a higher training throughput, and, unlike adapters, *no additional inference latency*. We also provide an empirical investigation into rank-deficiency in language model adaptation, which sheds light on the efficacy of LoRA. We release a package that facilitates the integration of LoRA with PyTorch models and provide our implementations and model checkpoints for RoBERTa, DeBERTa, and GPT-2 at <https://github.com/microsoft/LoRA>.

LoRA: la hipótesis

Sea un modelo preentrenado $P_{\Phi}(y|x)$. Lo queremos adaptar a una tarea downstream de generación condicional: resumen, comprensión lectora, NL2SQL, Etc.

Cada tarea es un dataset etiquetado $Z = \{(x_i, y_i)\}_{i=1..N}$

Fine-tuning completo. Se arranca en los pesos Φ_0 y se actualiza a $\Phi_0 + \Delta\Phi$ siguiendo el gradiente del objetivo de siempre de maximizar el token, condicionado en:

$$\max_{\Phi} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})) \quad P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y)$$

El problema. Por cada tarea se aprende un conjunto de $\Delta\Phi$ distinto cuya dimension $|\Delta\Phi| = |\Phi_0|$. Con GPT-3, eso es **175B de parámetros por tarea approx**. Guardar y desplegar muchas instancias independientes es inviable.

LoRA: la hipótesis

La idea. Reparametrizar el incremento: $\Delta\Phi = \Delta\Phi(\Theta)$, encodificandolo en un set de parámetros mas pequeño Θ con $|\Theta| \ll |\Phi_0|$. Buscar $\Delta\Phi$ pasa a ser optimizar sobre Θ :

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})) \quad \Rightarrow \quad \max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t}))$$

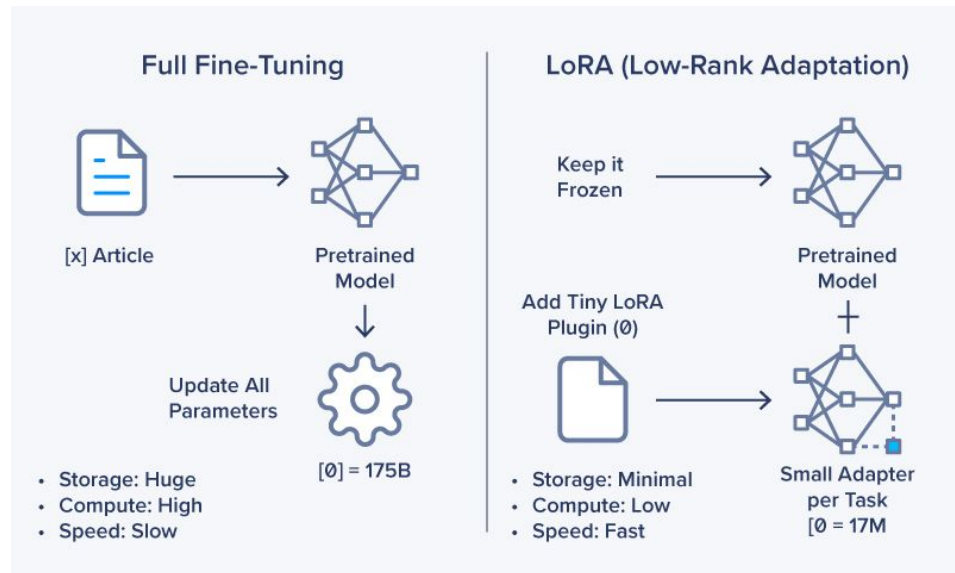
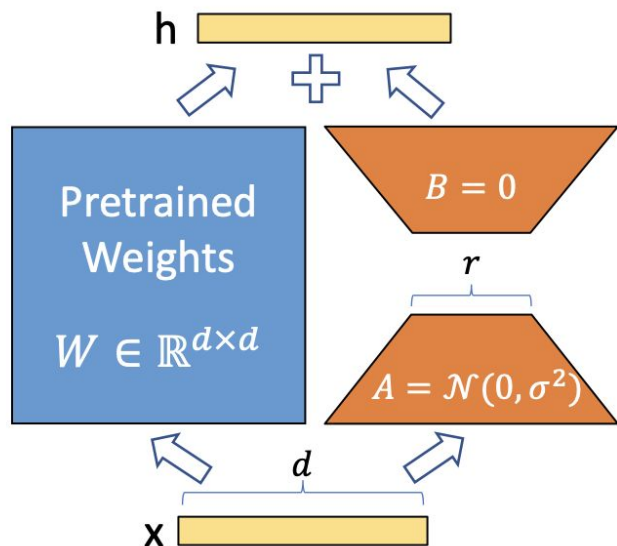
El resultado. Con una representación de rango bajo para codificar $\Delta\Phi$ sobre GPT-3 175B, $|\Theta|$ puede ser el **0,01% de $|\Phi_0|$** .

Lora in depth

<https://huggingface.co/learn/llm-course/chapter11/4>

LoRA: la hipótesis

Sobre GPT-3 175B: **10.000x menos parámetros entrenables** y **3x menos memoria de GPU**, con calidad igual o mejor que el fine-tuning completo



Variantes y el mundo real

- **QLoRA** (Dettmers, 2023) — base en 4 bits (NF4) + adapters en 16: un modelo de 65B fine-tuneado en **una sola GPU de 48 GB**
- **peft**: LoraConfig(r=8, lora_alpha=16, target_modules=[...]) y listo
- **Servir adapters**: pesan MB, no GB → muchos clientes, un solo modelo base en memoria
- **Fine-tuning gestionado** (OpenAI, HF AutoTrain): pagás el entrenamiento **y** un precio por token más alto al inferir

[Are you still using lora to fine-tune your llm?](#)

TP-2: de donde viene y adonde va

TP-1	modelo subword (tokenizer GPT-2)	—>	checkpoint
TP-2	SFT + LoRA sobre ese checkpoint	—>	checkpoints/tp2_full_ft
Clase 5	ese modelo servido en llama.cpp		

- Si no tienen el checkpoint del TP-1, el TP-2 lo preentrena solo. Nadie queda trabado
- **Guardar los checkpoints:** la clase 5 los usa

Referencias

- LoRA: Low-Rank Adaptation of Large Language Models — Hu et al., 2021
- Intrinsic Dimensionality Explains the Effectiveness of LM Fine-Tuning — Aghajanyan et al., 2020
- QLoRA: Efficient Finetuning of Quantized LLMs — Dettmers et al., 2023
- Improving Language Understanding by Generative Pre-Training (GPT-1) — Radford et al.
- Scaling Laws for Neural Language Models — Kaplan et al.
- Bias and Fairness in LLMs: A Survey — Gallegos et al.
- Will we run out of data? — Villalobos et al.
- Docs: HuggingFace Hub · PEFT · OpenAI API reference · Artificial Analysis